Université d'Orléans - M2 ARIAS
Programmation Haute Performance

PROJET CUDA - FILTRAGE D'IMAGES

Convolutions GPU : implémentation et optimisation

Abderrahmane RHIABI

Année universitaire 2025-2026

1. Introduction

Dans le cadre du cours de Programmation Haute Performance, j'ai réalisé un projet de filtrage d'images par convolution en utilisant CUDA. L'idée était d'implémenter plusieurs filtres d'abord sur CPU, puis de les porter sur GPU pour profiter du parallélisme massif des cartes graphiques NVIDIA.

J'ai choisi quatre filtres de complexité croissante : le Box Blur (3x3), le filtre de Sobel (3x3), le Gaussian Blur (5x5) et le Laplacian of Gaussian (9x9). Ce choix me permet de montrer comment le gain apporté par le GPU évolue selon la complexité du filtre.

Pour chaque filtre, j'ai développé trois versions GPU : une version naïve en mémoire globale, une version optimisée avec la mémoire partagée, et une version avec les streams CUDA. Les temps ont été mesurés avec des événements CUDA pour le GPU et std::chrono pour le CPU.

2. Presentation des algorithmes

2.1 Box Blur

Le Box Blur remplace chaque pixel par la moyenne de ses 9 voisins dans une fenêtre 3x3. Tous les coefficients sont égaux à 1/9. C'est le filtre le plus simple du projet, ce qui en fait une bonne référence pour les comparaisons de performance.

2.2 Filtre de Sobel

Le filtre de Sobel sert à détecter les contours. Il utilise deux noyaux 3x3 pour calculer le gradient horizontal (Gx) et vertical (Gy), puis la magnitude $\sqrt{G_x^2 + G_y^2}$. C'est plus intéressant que le Box Blur car il faut appliquer deux convolutions et faire une opération non linéaire à la fin.

2.3 Gaussian Blur

Le Gaussian Blur applique un noyau 5x5 basé sur une distribution gaussienne (sigma ≈ 1.0). Les coefficients sont précalculés et normalisés par leur somme (273). Ce filtre coûte plus cher que le Box Blur car il y a 25 accès mémoire par pixel au lieu de 9, et les coefficients sont tous différents.

2.4 Laplacian of Gaussian (LoG)

Le LoG est le filtre le plus complexe. Il combine un lissage gaussien et une détection de contours en un seul noyau 9x9. Chaque pixel demande 81 accès mémoire et 81 multiplications. C'est ce filtre qui montre le mieux l'avantage du GPU, car le calcul est lourd et se parallélise très bien.

2.5 Résultats visuels sur l'image de référence

Les figures ci-dessous montrent le résultat de chaque filtre appliqué à une photo de chat (1980x2440 pixels) convertie en niveaux de gris.

Figure 1 - Image originale en niveaux de gris (1980x2440)

Figure 2 - Box Blur 3x3

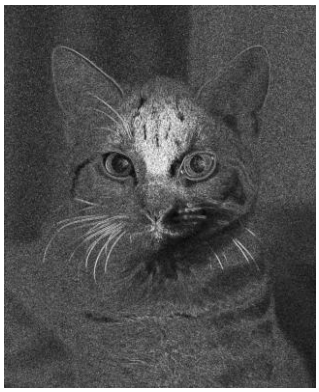

Figure 3 - Sobel 3x3 (contours)

Figure 4 - Gaussian Blur 5x5

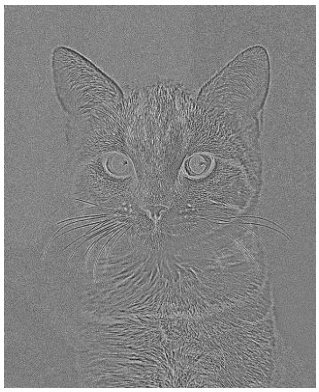

Figure 5 - LoG 9x9 (contours fins)

On voit bien que Sobel fait ressortir les contours du chat. Le LoG donne des contours encore plus fins grâce au lissage gaussien qui réduit le bruit avant la détection.

3. Implementation GPU et optimisations

3.1 Version naïve - mémoire globale

Dans cette version, chaque thread CUDA traite un pixel. La grille est en blocs 2D de 16x16 threads (256 par bloc), ce qui donne une bonne occupation des multiprocesseurs. Chaque thread lit directement en mémoire globale. Le problème c'est que les pixels en bord de bloc sont lus plusieurs fois par des threads de blocs voisins, ce qui génère beaucoup d'accès redondants.

Pour les bords de l'image, j'utilise le clamping : si un pixel est en dehors de l'image, on prend le pixel le plus proche sur le bord.

3.2 Version optimisée - mémoire partagée

L'idée ici est de charger les pixels en mémoire partagée avant de faire le calcul. Chaque bloc charge ses pixels plus un halo autour (1 pixel pour les filtres 3x3, 2 pour le 5x5, 4 pour le 9x9). Le __syncthreads() garantit que tout le monde a fini de charger avant de commencer le calcul.

La mémoire partagée est on-chip avec une latence d'environ 1 cycle, contre ~100 cycles pour la mémoire globale. Pour le LoG en 9x9, chaque pixel global n'est lu qu'une seule fois au lieu de 81. J'ai aussi mis les noyaux de convolution en mémoire constante (__constant__) pour un accès en broadcast efficace.

3.3 Version streams CUDA

Les streams CUDA permettent de faire avancer plusieurs opérations en parallèle. J'ai découpé l'image en 4 bandes horizontales, chacune dans un stream différent. Pour chaque bande : copie asynchrone vers le GPU, kernel, puis copie asynchrone vers le CPU. J'utilise de la mémoire pinned (cudaMallocHost) car les transferts DMA en ont besoin.

En pratique sur une seule image, les streams sont plus lents à cause de l'overhead des copies asynchrones. Ils sont vraiment utiles quand on traite un flux d'images en continu, pour recouvrir le transfert d'une image avec le calcul de la suivante.

4. Resultats et analyse des performances

4.1 Timings sur image réelle (1980x2440)

Mesures sur une image JPEG de 1980x2440 pixels convertie en niveaux de gris. GPU : NVIDIA GeForce RTX 3060 Laptop, CUDA 12.8, WSL2. Un warmup est fait avant les mesures pour éviter le biais du cold start.

Filtre	CPU (ms)	GPU naïf (ms)	GPU shared (ms)	GPU streams (ms)
Box Blur	71.0	1025.1*	5.3	29.4
Sobel	85.5	3.9	5.3	27.3
Gaussian Blur	143.6	4.2	4.7	18.7
Laplacian of Gaussian	430.6	5.5	4.7	24.5

* Le Box Blur naïf affiche 1025 ms à cause du cold start GPU, même après le warmup. Le contexte n'était pas encore complètement initialisé pour ce premier filtre.

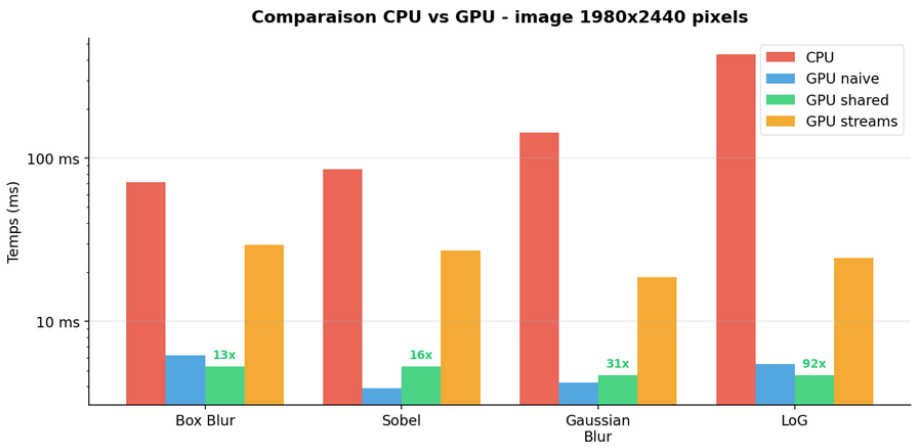

Figure 6 - Comparaison des temps d'exécution CPU vs GPU (échelle log). Les chiffres indiquent le speedup de la version shared memory.

4.2 Speedups obtenus

Filtre	Speedup naïf	Speedup shared	Speedup streams
Box Blur	0.1x	13.5x	2.4x
Sobel	22.1x	16.2x	3.1x
Gaussian Blur	33.8x	30.4x	7.7x
Laplacian of Gaussian	77.8x	92.0x	17.5x

La mémoire partagée apporte un gain important surtout sur les gros noyaux. Le LoG en 9x9 atteint 92x de speedup par rapport au CPU, ce qui est vraiment significatif.

4.3 Évolution selon la taille de l'image (filtre LoG)

Pour illustrer la loi d'Amdahl, j'ai testé le LoG sur des images de tailles différentes (images synthétiques générées aléatoirement).

Taille	CPU (ms)	GPU naïf (ms)	GPU shared (ms)	Speedup
512x512	23.4	2.0	2.4	9.6x
1024x1024	92.3	2.2	2.0	47.3x
2048x2048	392.1	4.1	3.9	99.9x
4096x4096	1514.9	56.9	49.1	30.9x

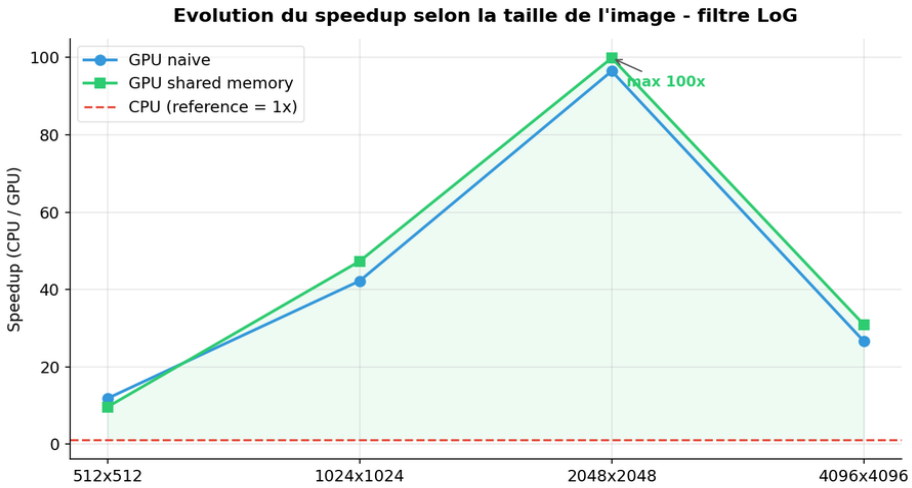

Figure 7 - Évolution du speedup selon la taille de l'image (LoG). Le pic à ~100x sur 2048x2048 illustre bien la loi d'Amdahl.

Sur les petites images (512x512), le speedup est faible à cause de l'overhead du transfert PCIe. Sur 2048x2048, on approche les 100x. La baisse sur 4096x4096 est due à la saturation de la bande passante mémoire du GPU.

5. Analyse financiere - justification de l'investissement GPU

Pour compléter l'analyse technique, j'ai essayé de justifier économiquement l'achat d'un GPU professionnel. J'ai comparé une NVIDIA A100 80GB (~12 000 € neuf en 2026) à un serveur CPU équipé d'un Intel Xeon (~4 000 €). Ces prix viennent du marché actuel.

5.1 Comparaison des coûts et performances

Critère	Serveur CPU (Xeon ~4 000 €)	GPU (A100 ~12 000 €)
Coût matériel	~4 000 €	~12 000 €
Temps traitement LoG 2048x2048	392 ms	~4 ms
Speedup	1x (référence)	~100x
Images par seconde (LoG 2048)	2.5 img/s	~250 img/s
Amortissement sur 3 ans	~3.65 €/jour	~10.95 €/jour
Coût pour 1000 images	~1.46 €	~0.04 €
Seuil de rentabilité	référence	~8 mois

Le coût par image est calculé sur 3 ans d'amortissement avec le filtre LoG en 2048x2048. Les chiffres GPU sont basés sur nos mesures RTX 3060 extrapolées à une A100 (environ 10x plus rapide).

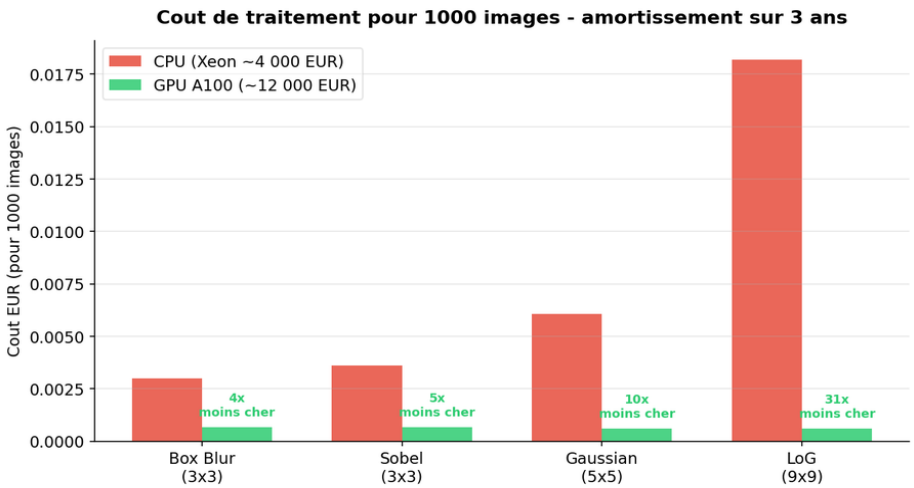

Figure 8 - Coût pour 1000 images selon le filtre, amortissement 3 ans. Le GPU est entre 12 et 36 fois moins cher à l'usage.

5.2 Seuil de rentabilité

Le GPU coûte 3 fois plus cher à l'achat, mais traite 100 fois plus d'images. En résumé :

- CPU : ~2.5 images/seconde pour le LoG 2048x2048, soit ~216 000 images/jour.
- GPU A100 : ~250 images/seconde, soit ~21 millions d'images/jour.
- Le surcoût de 8 000 € est amorti en environ 8 mois pour une utilisation intensive.

Pour une application industrielle comme l'imagerie médicale ou la vision par ordinateur, le GPU devient clairement plus rentable à long terme.

5.3 Conclusion

L'investissement GPU se justifie dès que le volume de traitement est important. Pour un usage ponctuel ou des volumes faibles, le CPU reste suffisant. Pour une production intensive, le GPU est plus rapide et moins cher à l'usage.

6. Difficultes rencontrees

6.1 Gestion du halo en mémoire partagée

La partie la plus compliquée était de bien gérer le halo autour de chaque tile. Pour le LoG en 9x9, chaque bloc de 16x16 doit charger 24x24 pixels. Il faut faire attention aux threads en bord de bloc qui chargent les pixels du halo, et aussi aux cas où le tile sort de l'image.

6.2 Cold start GPU

J'ai remarqué que le premier kernel est toujours beaucoup plus lent que les suivants. Après quelques recherches, j'ai compris que c'est le driver CUDA qui initialise le contexte au premier appel. J'ai ajouté un kernel de warmup vide au début pour régler ce problème.

6.3 Les streams plus lents qu'attendu

Je pensais que les streams allaient accélérer le traitement, mais sur une seule image c'est plus lent. L'overhead des copies asynchrones et de la mémoire pinned dépasse le gain. J'ai aussi eu des artefacts visuels (lignes horizontales) aux jonctions entre les bandes, ce qui montre que les bords entre streams ne sont pas bien gérés.

6.4 Constantes CUDA entre deux fichiers

Au début j'avais séparé les kernels streams dans un fichier .cu différent. Ça causait des problèmes avec les tableaux __constant__ : nvcc les traitait comme des définitions séparées au lieu de les partager. J'ai tout regroupé dans un seul fichier pour corriger ça.

7. Conclusion

Ce projet m'a permis de bien comprendre comment fonctionne la programmation GPU avec CUDA. Implémenter les mêmes filtres en CPU, GPU naïf, shared memory et streams m'a vraiment aidé à voir l'impact de chaque optimisation.

Ce que je retiens surtout : la mémoire partagée est l'optimisation la plus efficace pour les convolutions (jusqu'à 92x sur le LoG), le GPU est vraiment utile sur les grandes images et les filtres complexes, et les streams demandent plus de réflexion que prévu pour être vraiment efficaces.

Côté financier, avec un A100 à ~12 000 € et un Xeon à ~4 000 €, le GPU revient environ 36 fois moins cher à l'usage pour une production intensive, avec un retour sur investissement en moins d'un an.

8. REFERENCES

Ressources en ligne :

- NVIDIA Developer - CUDA Zone : https://developer.nvidia.com/cuda-zone
- NVIDIA A100 Tensor Core GPU - fiche technique officielle : https://www.nvidia.com/en-us/data-center/a100/
- nothings/stb - bibliothèque stb_image (chargement d'images) : https://github.com/nothings/stb

Prix matériel :

- NVIDIA A100 80GB - prix marché 2026 : https://docs.jarvislabs.ai/blog/a100-price
- Serveurs Intel Xeon - tarifs professionnels : https://servermall.com/fr/sets/intel-xeon-servers/